CLIP FLIP BLIP SigLIP. . .

Introduction to CLIP Variants

Oct 7, 2026

AID 7기 이치오

Learning Transferable Visual Models From Natural Language Supervision

Radford et al., 2021, # of citations: 75060

https://arxiv.org/abs/2103.00020

Abstract, Introduction

(1) SoTA methods depend on the fixed set of predetermined obj. categories

→ Learning from the raw text about images

(2) Which caption goes to w/ which image is an efficient and scalable

(3) Transfers to downstream tasks non-trivially

Data collection

(1) Crawl Wikipedia for all words occurring at least 100 times + Page titles
(2) Search for images for above queries
(3) Balance the results by including up to 20k per a query

Contrastive Learning

(1) Training a system to work on the proxy task of predicting which text matches which image is potentially easier

(2) Min. the cosine sim. for N^2 - N (incorrect pairing)

(3) Max. the cosine sim. for N (correct!)

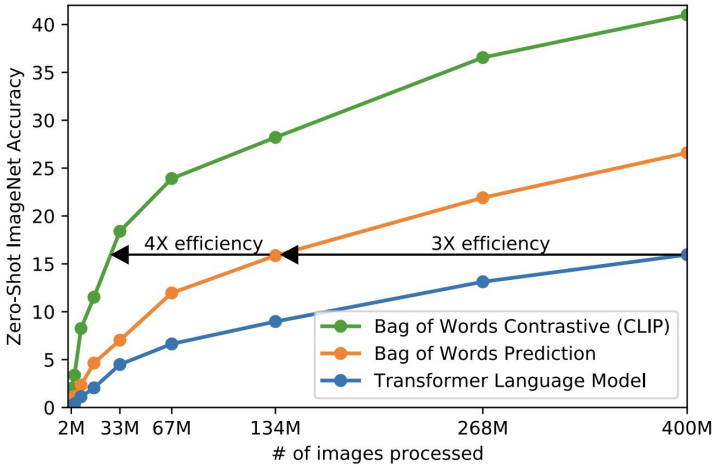


Figure 2. **CLIP is much more efficient at zero-shot transfer than our image caption baseline.** Although highly expressive, we found that transformer-based language models are relatively weak at zero-shot ImageNet classification. Here, we see that it learns 3x slower than a baseline which predicts a bag-of-words (BoW) encoding of the text (Joulin et al., 2016). Swapping the prediction objective for the contrastive objective of CLIP further improves efficiency another 4x.

Model Arch.

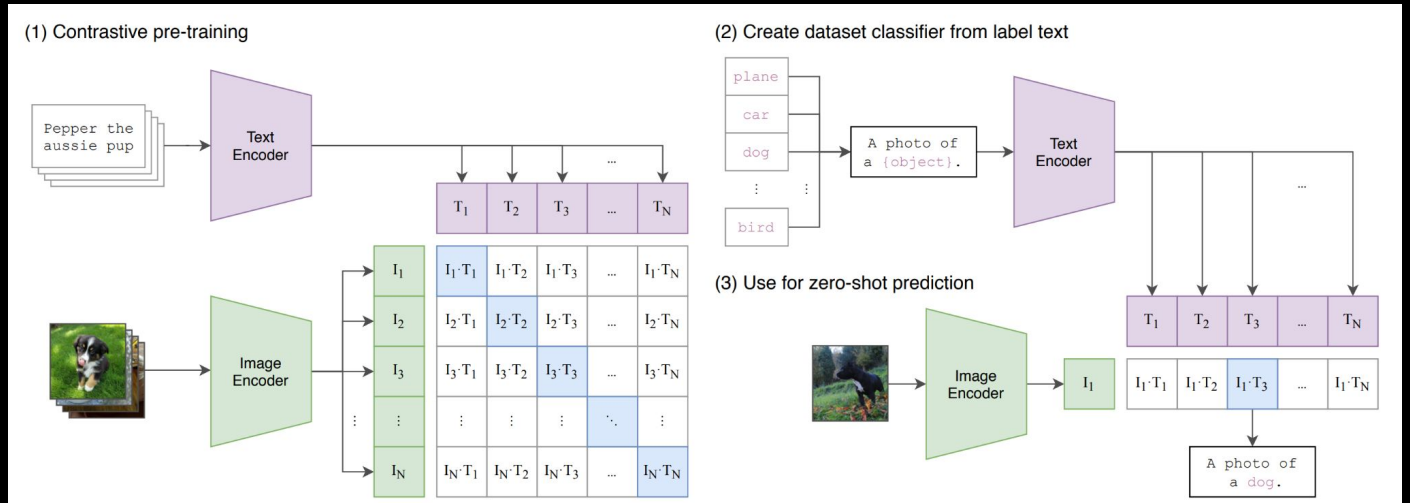


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

Additional methodologies and loss formulation

(1) The only DA done is simple rand. sq. crop
(2) Use learnable temp. param. t to control the range of softmax
(3) t is optimized as a log-parametrized multiplicative scalar

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

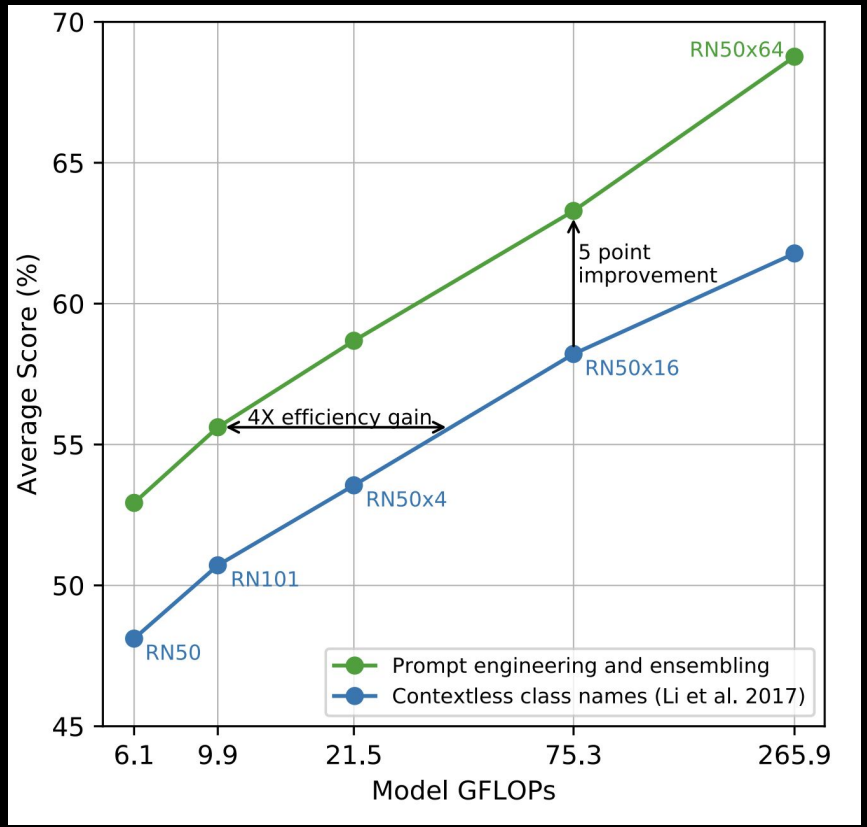
Prompt Engineering

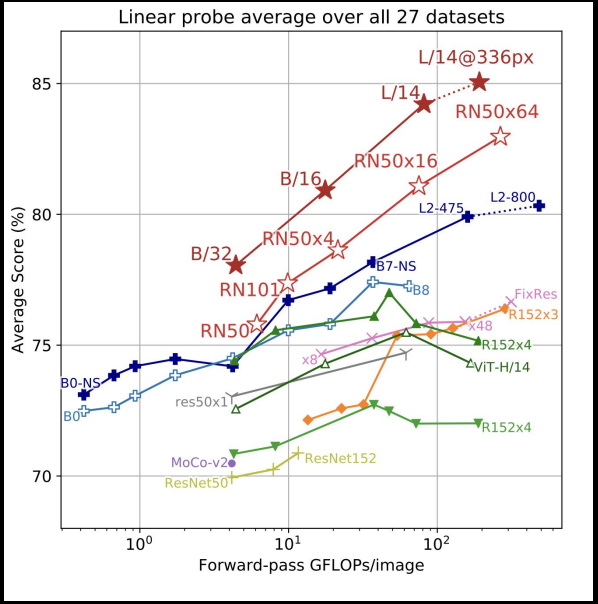
(1) A photo of a {}
(2) A satellite photo of a {}

.

.

.





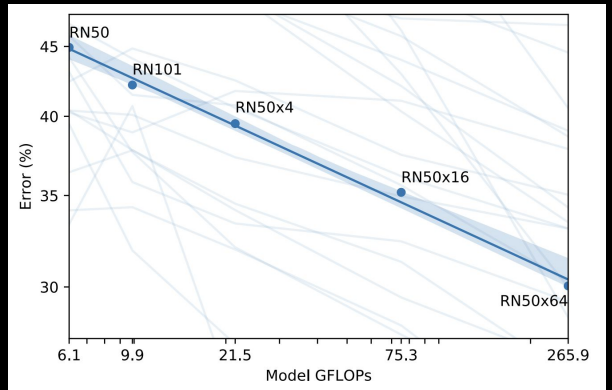
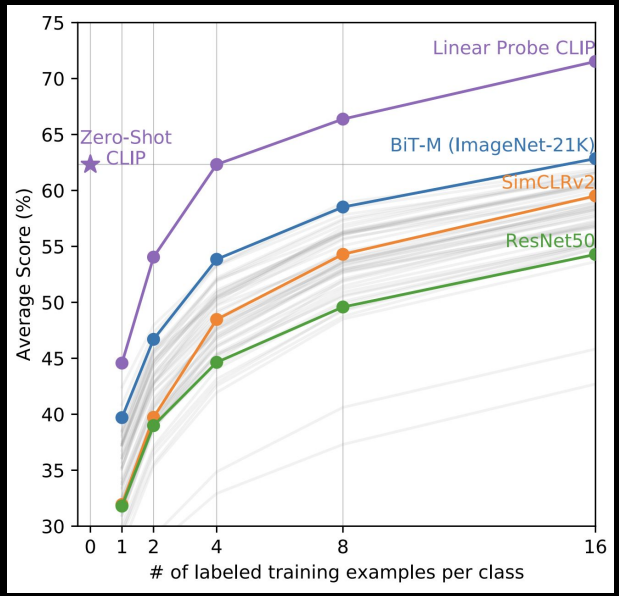


Figure 9. **Zero-shot CLIP performance scales smoothly as a function of model compute.** Across 39 evals on 36 different datasets, average zero-shot error is well modeled by a log-log linear trend across a 44x range of compute spanning 5 different CLIP models. Lightly shaded lines are performance on individual evals, showing that performance is much more varied despite the smooth overall trend.



Scaling law discovery, catastrophic forgetting induced by few-shot linear probing

Scaling Language-Image Pre-training via Masking

Li et al., 2022, # of citations: 517

https://arxiv.org/abs/2212.00794

MAE Abstract, Introduction (Crash Course)

Rationales:

(1) ViT = Data-hungry

(2) GPT: Autoregressive Modeling

(3) BERT: Masked Autoencoding

(4) Image has heavy spatial redundancy

(mask can be reversed w/ extrapolation)

https://arxiv.org/abs/2111.06377

MAE Abstract, Introduction (Crash Course)

Methodologies:

(1) Mask image patches

→ Reconstruct

(2) Asymmetric Encoder-Decoder

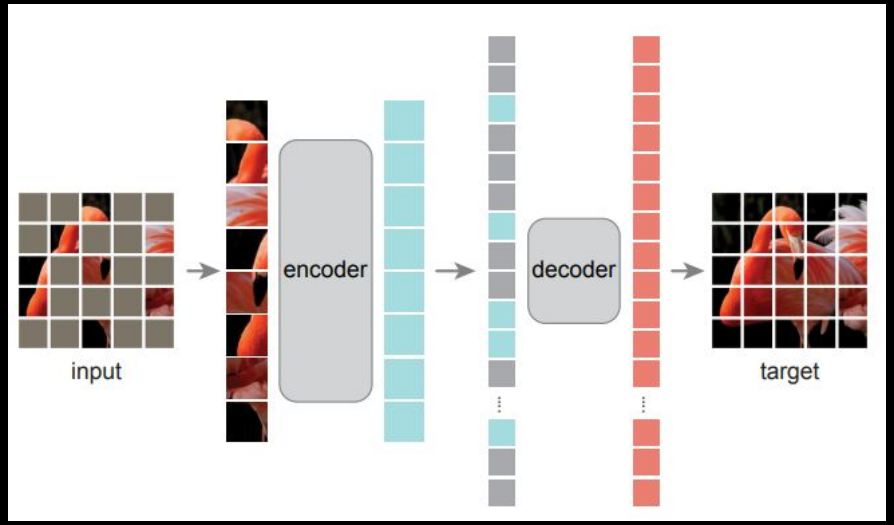
(smaller decoder)

Findings:

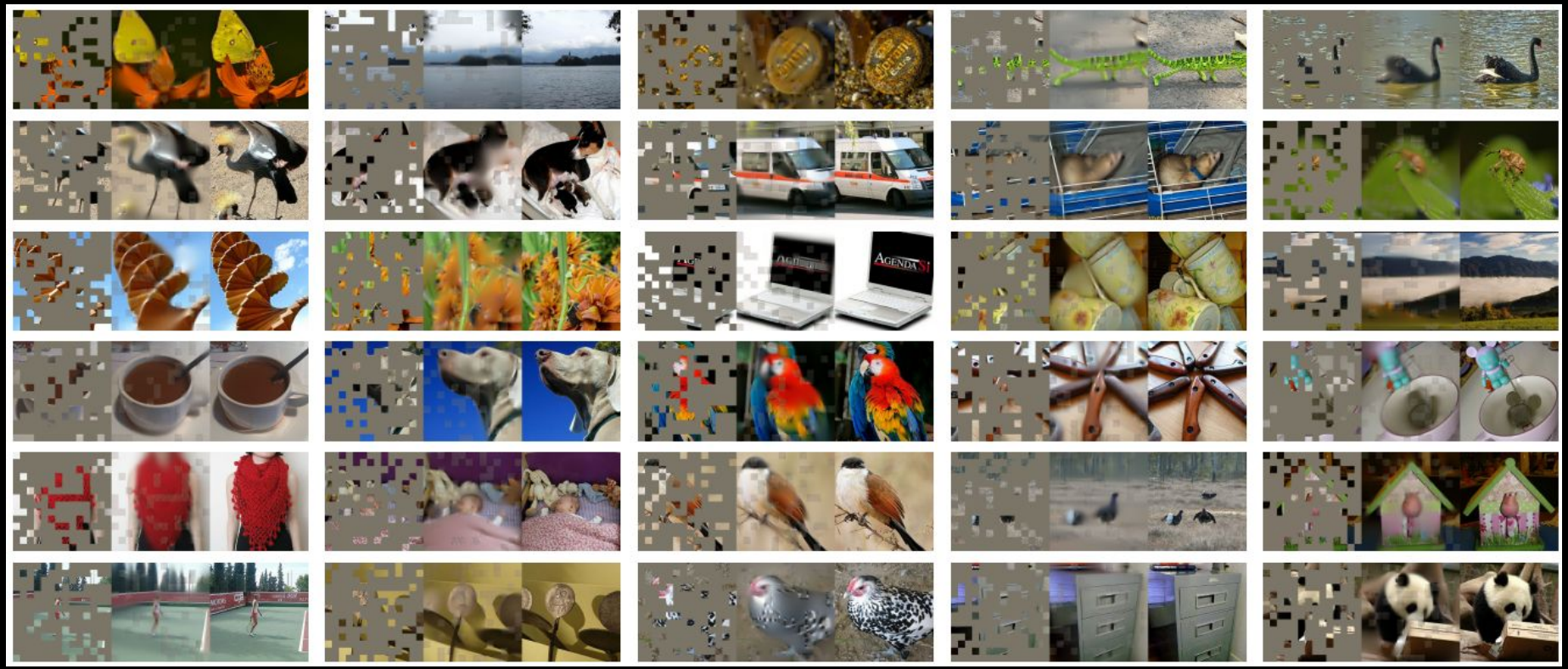
(1) Masking significant portion of image

→ nontrivial performance increase

(2) Training Acceleration (> 3x)



MAE Approach: Results



MAE Questions

Q. Use of smaller decoder?

In vision, the decoder reconstructs pixels, hence its output is of a lower semantic level than common recognition tasks; different from BERT, where the decoder predicts missing words that contain rich semantic information.

Q. Use of mask tokens in decoder input?

If the encoder uses mask tokens, it performs worse. The encoder has a large portion of mask tokens in its input in pretraining, which does not exist in uncorrupted images. By skipping the mask token in the encoder, this may also reduce training computation.

FLIP Arch.

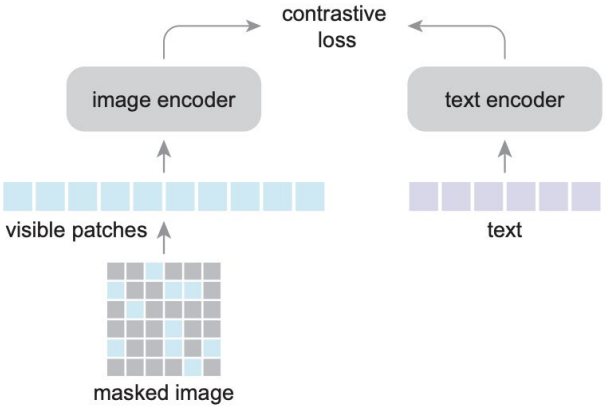


Figure 2. **Our FLIP architecture**. Following CLIP [52], we perform contrastive learning on pairs of image and text samples. We randomly mask out image patches with a high masking ratio and encode only the visible patches. We do not perform reconstruction of masked image content.

FLIP Speed-up

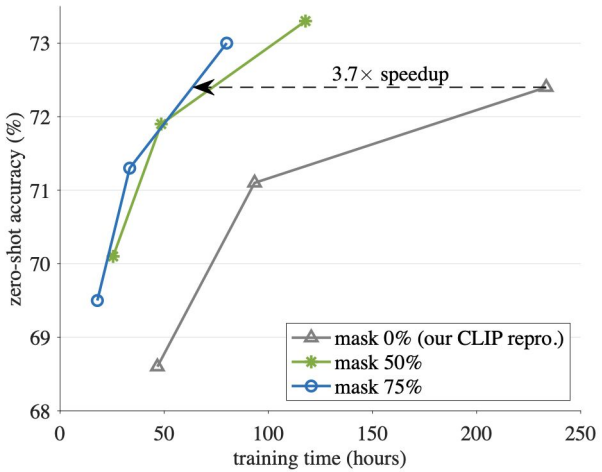


Figure 1. **Accuracy *vs.* training time trade-off**. With a high masking ratio of 50% or 75%, our FLIP method trains faster and is more accurate than its CLIP counterpart. All entries are benchmarked in 256 TPU-v3 cores. Training is done on LAION-400M for 6.4, 12.8, or 32 epochs, for each masking ratio. Accuracy is evaluated by zero-shot transfer on the ImageNet-1K validation set. The model is ViT-L/16 [20]. More details are in Fig. 3. As the CLIP baseline takes ~2,500 TPU-days training, a speedup of 3.7× can save ~1,800 TPU-days.

FLIP Ablation study

mask	batch	FLOPs	time	acc.
0%	16k	1.00×	1.00×	68.6
50%	32k	0.52×	0.50×	**69.6**
75%	64k	0.28×	0.33×	68.2

(a) **Image masking** yields higher or comparable accuracy and speeds up training. Entries are subject to the same memory limit.

batch	mask 50%	mask 75%
16k	68.5	65.8
32k	69.6	67.3
64k	**70.4**	**68.2**

(b) **Batch size**. A large batch has big gains over smaller batches.

text mask	text len	time	acc.
baseline, 0%	32	1.00×	**68.2**
random, 50%	16	0.92×	66.0
prioritized, 50%	16	0.92×	67.8

(c) **Text masking** performs decently, but the speed gain is marginal as its encoder is smaller. Here the image masking ratio is 75%.

	mask 50%	mask 75%
w/ mask	66.4	60.9
w/ mask, ensemble	68.1	65.1
w/o mask	**69.6**	**68.2**

(d) **Inference unmasking**. Inference on intact images performs strongly even without tuning.

	mask 50%	mask 75%
baseline	69.6	68.2
+ tuning	**70.1**	**69.5**

(e) **Unmasked tuning**. The distribution shift by masking is reduced by a short tuning.

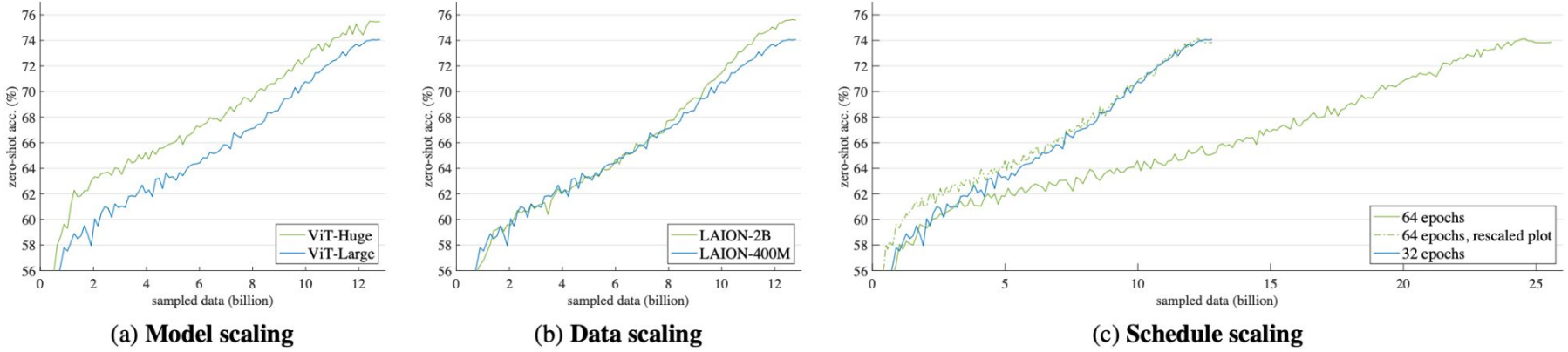
	mask 50%	mask 75%
baseline	**69.6**	**68.2**
+ MAE	69.4	67.9

(f) **Reconstruction**. Adding the MAE reconstruction loss has no gain.

Table 1. **Zero-shot ablation experiments**. Pre-training is on LAION-400M for 6.4 epochs, evaluated by zero-shot classification accuracy on ImageNet-1K validation. The backbone is ViT-L/16 [20]. Unless specified, the baseline setting is: image masking is 50% (batch 32k) or 75% (batch 64k), text masking is 0%, and no unmasked tuning is used.

Masked pre-training requires unmasked tuning for the input distribution alignment

FLIP further scaling directions



(a) **Model scaling** (b) **Data scaling** (c) **Schedule scaling**

Figure 4. **Training curves of scaling**. The x-axis is the number of sampled data during training, and the y-axis is the zero-shot accuracy on IN-1K. The blue curve is the baseline setting: ViT-Large, LAION-400M data, 32 epochs (12.8B sampled data). In each subplot, we compare with scaling one factor on the baseline. In schedule scaling (Fig. 4c), we plot an extra hypothetical curve for a better visualization.

BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation

Li et al., 2022, # of citations: 10693

https://arxiv.org/abs/2201.12086

BLIP Arch.

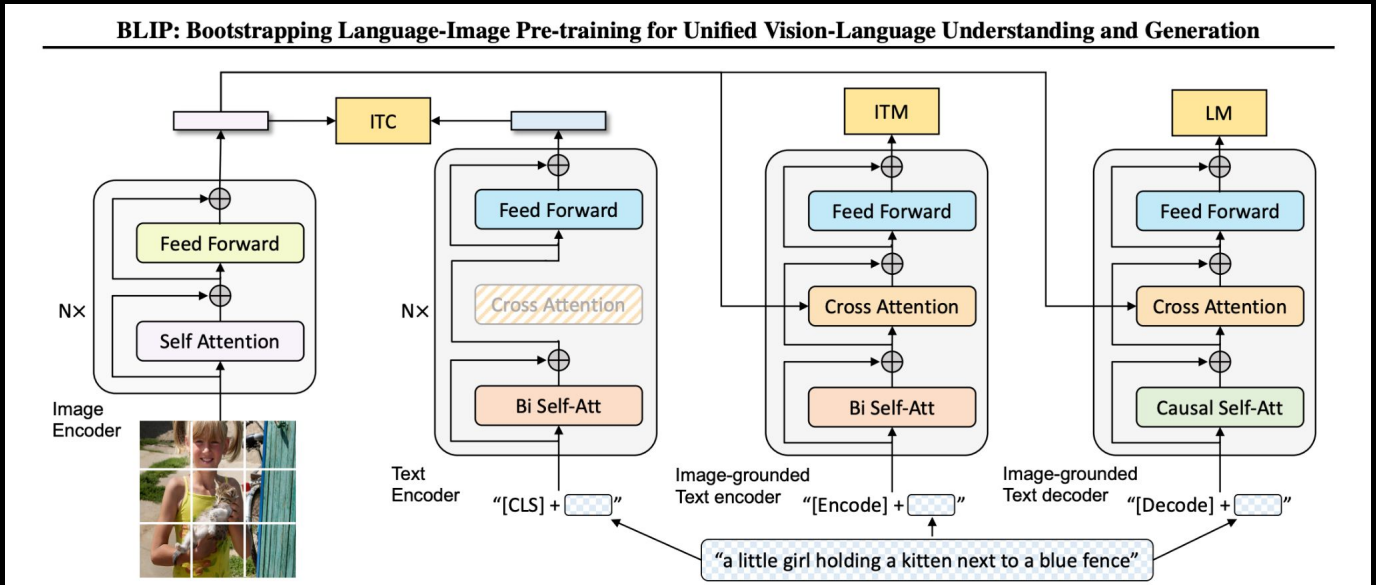


Figure 2. Pre-training model architecture and objectives of BLIP (same parameters have the same color). We propose multimodal mixture of encoder-decoder, a unified vision-language model which can operate in one of the three functionalities: (1) Unimodal encoder is trained with an image-text contrastive (ITC) loss to align the vision and language representations. (2) Image-grounded text encoder uses additional cross-attention layers to model vision-language interactions, and is trained with a image-text matching (ITM) loss to distinguish between positive and negative image-text pairs. (3) Image-grounded text decoder replaces the bi-directional self-attention layers with causal self-attention layers, and shares the same cross-attention layers and feed forward networks as the encoder. The decoder is trained with a language modeling (LM) loss to generate captions given images.

BLIP CapFlt

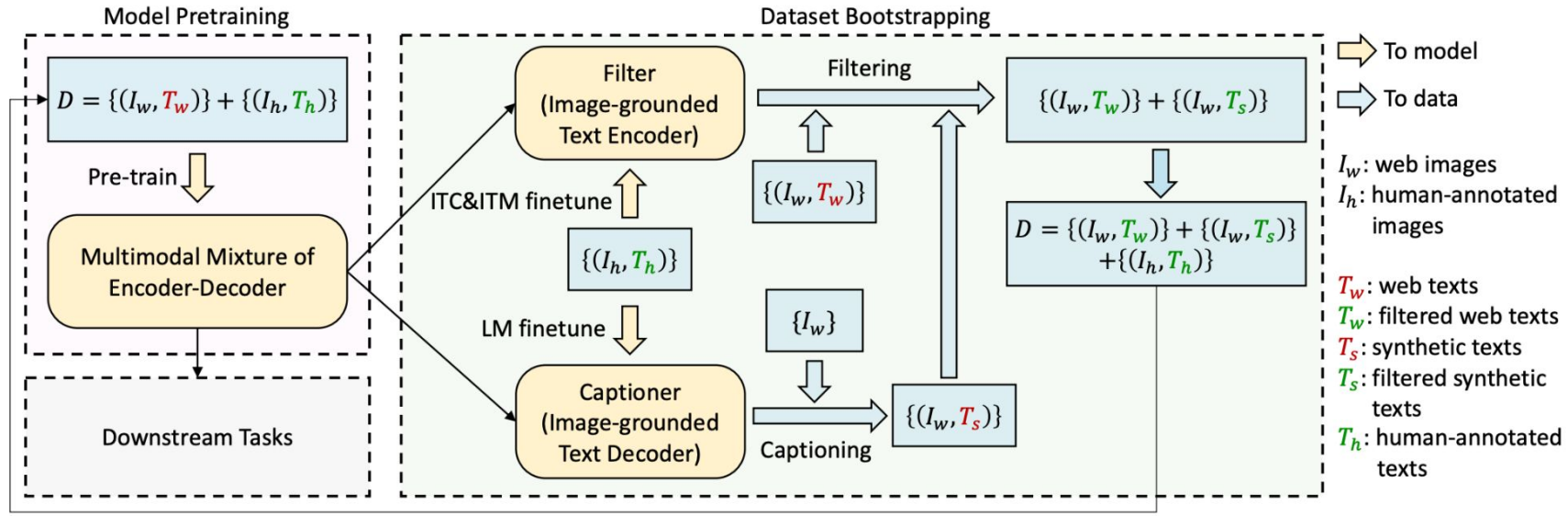


Figure 3. Learning framework of BLIP. We introduce a captioner to produce synthetic captions for web images, and a filter to remove noisy image-text pairs. The captioner and filter are initialized from the same pre-trained model and finetuned individually on a small-scale human-annotated dataset. The bootstrapped dataset is used to pre-train a new model.

BLIP Ablation study

Generation method	Noise ratio	Retrieval-FT (COCO) TR@1	Retrieval-FT (COCO) IR@1	Retrieval-ZS (Flickr) TR@1	Retrieval-ZS (Flickr) IR@1	Caption-FT (COCO) B@4	Caption-FT (COCO) CIDEr	Caption-ZS (NoCaps) CIDEr	Caption-ZS (NoCaps) SPICE
None	N.A.	78.4	60.7	93.9	82.1	38.0	127.8	102.2	13.9
Beam	19%	79.6	61.9	94.1	83.1	38.4	128.9	103.5	14.2
Nucleus	25%	80.6	63.1	94.8	84.9	38.6	129.7	105.1	14.4

Table 2. Comparison between beam search and nucleus sampling for synthetic caption generation. Models are pre-trained on 14M images.

Layers shared	#parameters	Retrieval-FT (COCO) TR@1	Retrieval-FT (COCO) IR@1	Retrieval-ZS (Flickr) TR@1	Retrieval-ZS (Flickr) IR@1	Caption-FT (COCO) B@4	Caption-FT (COCO) CIDEr	Caption-ZS (NoCaps) CIDEr	Caption-ZS (NoCaps) SPICE
All	224M	77.3	59.5	93.1	81.0	37.2	125.9	100.9	13.1
All except CA	252M	77.5	59.9	93.1	81.3	37.4	126.1	101.2	13.1
All except SA	252M	78.4	60.7	93.9	82.1	38.0	127.8	102.2	13.9
None	361M	78.3	60.5	93.6	81.9	37.8	127.4	101.8	13.9

Table 3. Comparison between different parameter sharing strategies for the text encoder and decoder during pre-training.

Diversity (nucleus decoding?), training/force generalization on diff. tasks using shared modules

BLIP Ablation study (continued)

Continue	Retrieval-FT (COCO)		Retrieval-ZS (Flickr)		Caption-FT (COCO)		Caption-ZS (NoCaps)	
	TR@1	IR@1	TR@1	IR@1	B@4	CIDEr	CIDEr	SPICE
Yes	80.6	63.0	94.5	84.6	38.5	129.9	104.5	14.2
No	80.6	63.1	94.8	84.9	38.6	129.7	105.1	14.4

Table 13. Continue training the pre-trained model offers less gain compared to training a new model with the bootstrapped dataset.

Re-train, do not cont. learning

BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models

Li et al., 2023, # of citations: 14769

https://arxiv.org/abs/2301.12597

Abstract

(1) Frozen image encoder
(2) Frozen LLM
(3) Q-former
    (a) Learnable query set

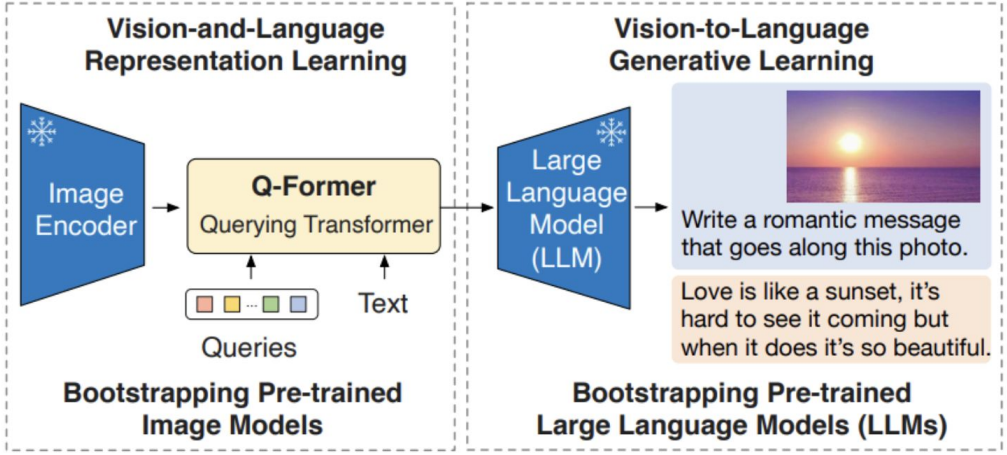


Figure 1. Overview of BLIP-2's framework. We pre-train a lightweight Querying Transformer following a two-stage strategy to bridge the modality gap. The first stage bootstraps vision-language representation learning from a frozen image encoder. The second stage bootstraps vision-to-language generative learning from a frozen LLM, which enables zero-shot instructed image-to-text generation (see Figure 4 for more examples).

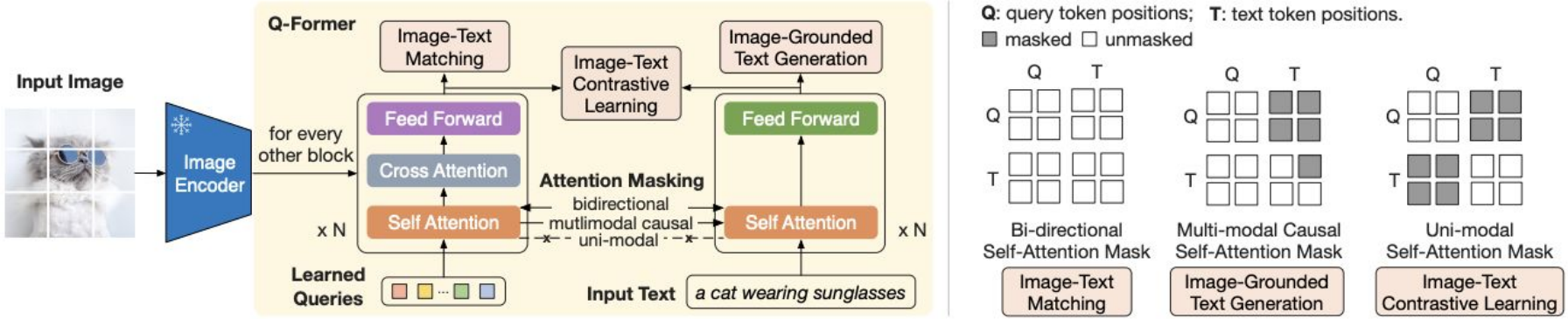


Figure 2. (**Left**) Model architecture of Q-Former and BLIP-2's first-stage vision-language representation learning objectives. We jointly optimize three objectives which enforce the queries (a set of learnable embeddings) to extract visual representation most relevant to the text. (**Right**) The self-attention masking strategy for each objective to control query-text interaction.

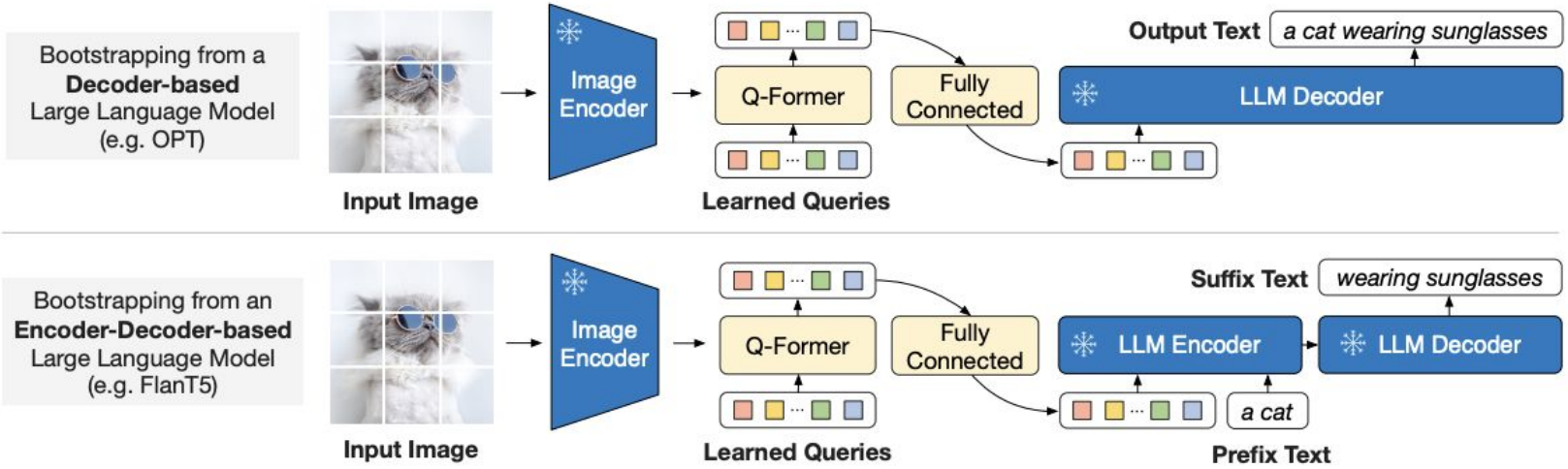


Figure 3. BLIP-2's second-stage vision-to-language generative pre-training, which bootstraps from frozen large language models (LLMs). (**Top**) Bootstrapping a decoder-based LLM (e.g. OPT). (**Bottom**) Bootstrapping an encoder-decoder-based LLM (e.g. FlanT5). The fully-connected layer adapts from the output dimension of the Q-Former to the input dimension of the chosen LLM.

Abstract, pros. & cons.

(1) Frozen image encoder
(2) Frozen LLM
(3) Q-former
    (a) Learnable query set
(4) Reduce min. required amount of data, low cost
(5) Cannot fully optimize the multimodel relation

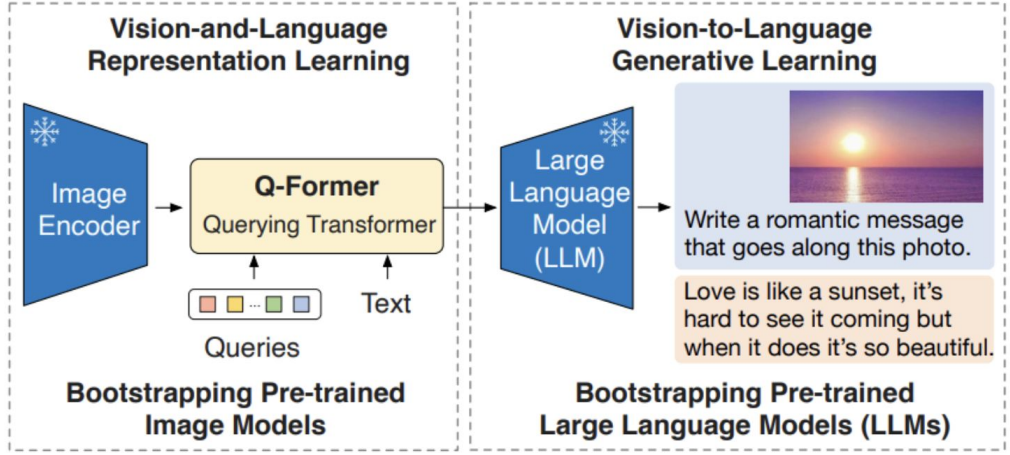


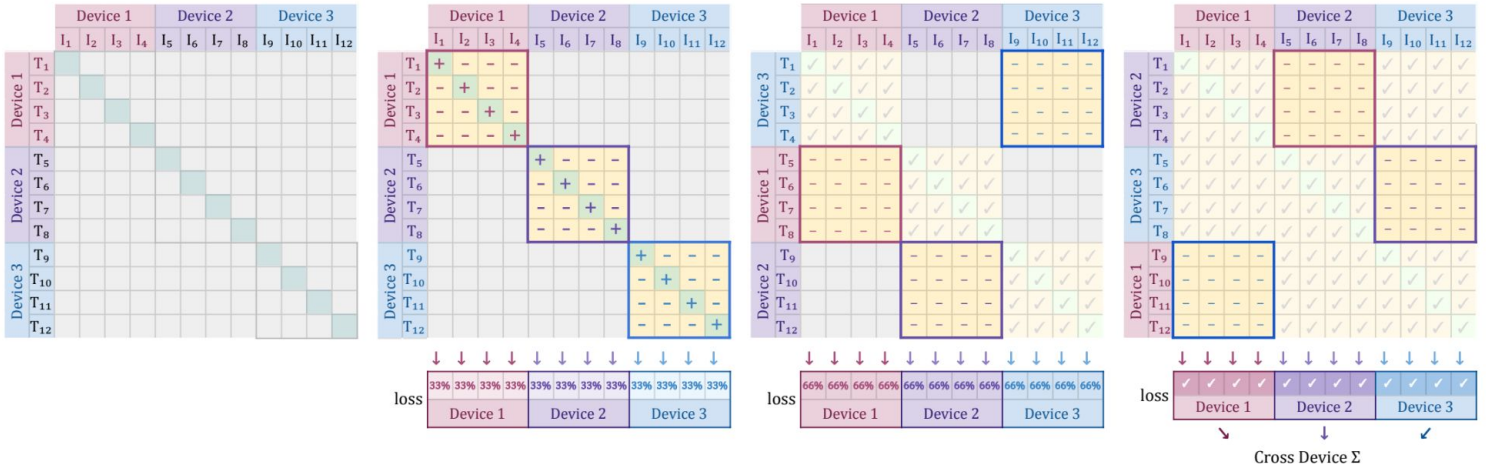
Figure 1. Overview of BLIP-2's framework. We pre-train a lightweight Querying Transformer following a two-stage strategy to bridge the modality gap. The first stage bootstraps vision-language representation learning from a frozen image encoder. The second stage bootstraps vision-to-language generative learning from a frozen LLM, which enables zero-shot instructed image-to-text generation (see Figure 4 for more examples).

Sigmoid Loss for Language Image Pre-Training

Zhai et al., 2023, # of citations: 4764

https://arxiv.org/abs/2303.15343

SigLip in a nutshell



(a) Initially each device holds 4 image and 4 text representations. Each device needs to see the representations from other devices to calculate the full loss.

(b) They each compute the component of the loss (highlighted) for their representations, which includes the positives.

(c) Texts are swapped across the devices, so device 1 now has $I_{1:4}$ and $T_{5:8}$ etc. The new loss is computed and accumulated with the previous.

(d) This repeats till every image & text pair have interacted, e.g. device 1 has the loss of $I_{1:4}$ and $T_{1:12}$. A final cross-device sum brings everything together.

Figure 1: **Efficient loss implementation** demonstrated via a mock setup with 3 devices and a global batch size of 12. There are no all-gathers, and at any point in time only the bright yellow square (size 4×4) is materialized in memory.

SigLip in a nutshell

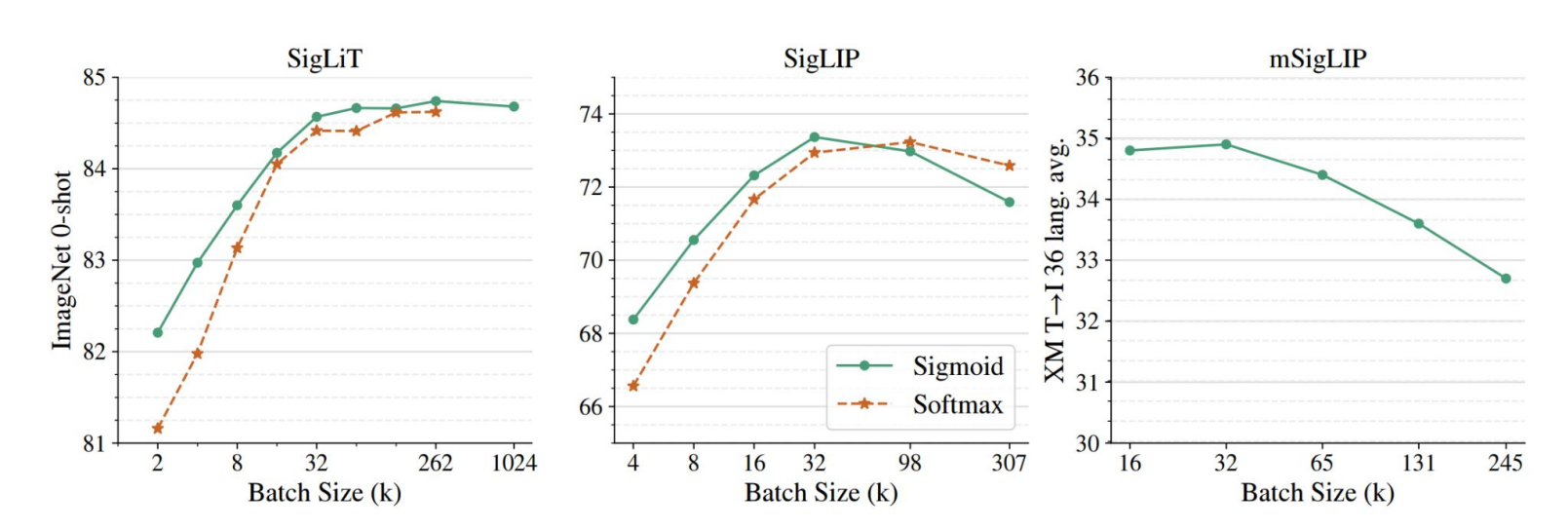
$$-\frac{1}{2|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\left(\overbrace{\log\frac{e^{t\mathbf{x}_i\cdot\mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|}e^{t\mathbf{x}_i\cdot\mathbf{y}_j}}}^{\text{image}\to\text{text softmax}}+\overbrace{\log\frac{e^{t\mathbf{x}_i\cdot\mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|}e^{t\mathbf{x}_j\cdot\mathbf{y}_i}}}^{\text{text}\to\text{image softmax}}\right)$$

from row-col two-step softmax application to the pairwise sigmoid normalization(?). may add bias b as sigmoid wouldn't cancel it. z_{ij} is the mask for the diag..

Consider bias b as the handicap for the pos. pair instances in the diag. due to the somewhat high influence of class imbalance in the earlier stage (1:16k, 1.6k, .)

$$-\frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\sum_{j=1}^{|\mathcal{B}|}\underbrace{\log\frac{1}{1+e^{z_{ij}(-t\mathbf{x}_i\cdot\mathbf{y}_j+b)}}}_{\mathcal{L}_{ij}}$$

SigLip in a nutshell



SigLip in a nutshell

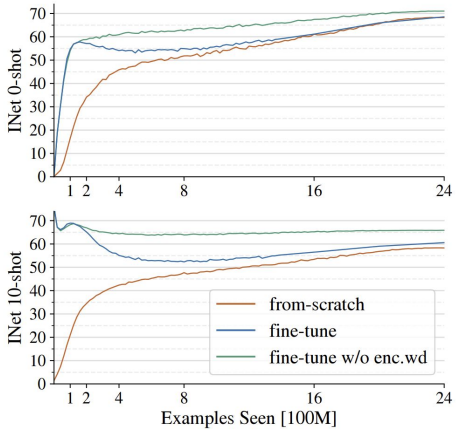


Figure 4: **Top**: SigLIP with pre-trained encoders ramps up quickly. However, only disabling weight decay on the pre-trained encoder weights leads to stable behavior and good ImageNet 0-shot transfer results. **Bottom**: ImageNet 10-shot transfer results, where decaying the pre-trained weights leads to deterioration of the pre-trained model visual representation quality. Disabling weight decay flattens the curve.

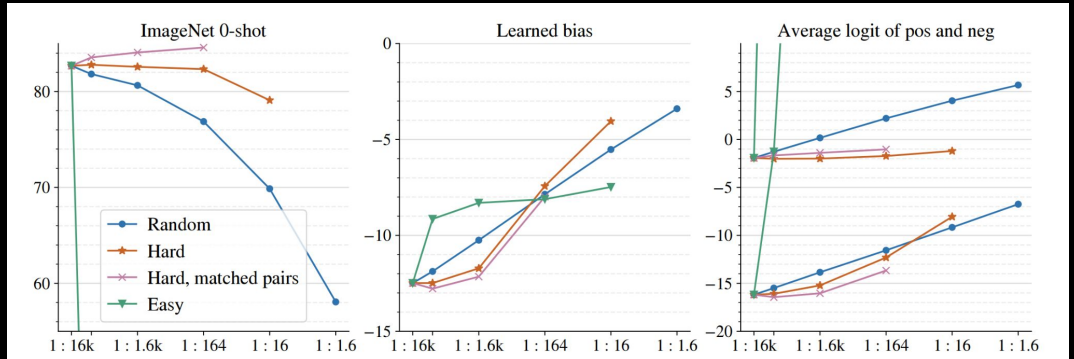


Figure 6: **The effect of batch composition.** We simulate various batch compositions by masking out negatives, either randomly, keeping only the hardest, or the easiest. With no masking, we have 16 k negatives for each positive in the batch (1:16 k) and the strongest masking we apply (1:1.6) results in almost balanced minibatches. In one setting we *match total pairs* seen by training for significantly longer. We observe ImageNet 0-shot score, the final value of the learned bias, and the average logits of positive and negative pairs. Overall, the imbalance does not seem to be detrimental, but finding an *efficient* way of mining negatives might be beneficial.

Use hard problems (match between dog, retriever, etc)

Emerging Properties in Self-Supervised Vision Transformers

Caron et al., 2021, # of citations: 13849

https://arxiv.org/abs/2104.14294

Abstract, Introduction

(1) Could the SSL provide new prop. to ViT stands out compared to CNN?
   (a) SSLed ViT features contain explicit info. about the semantic segmentation
   (b) above features are also excellent k-nn classifiers
(2) Use of momentum encoder, multi-crop training, and the small patch use
(3) Self-**di**stillation w/ **no** labels

$$P_s(x)^{(i)} = \frac{\exp(g_{\theta_s}(x)^{(i)}/\tau_s)}{\sum_{k=1}^{K} \exp(g_{\theta_s}(x)^{(k)}/\tau_s)}, \qquad (1)$$

$$\min_{\theta_s} H(P_t(x), P_s(x)), \qquad (2)$$

where $H(a, b) = -a \log b$.

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x')). \qquad (3)$$

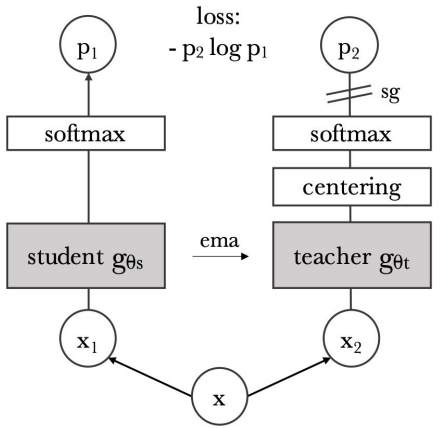


Figure 2: **Self-distillation with no labels.** We illustrate DINO in the case of one single pair of views (x_1, x_2) for simplicity. The model passes two different random transformations of an input image to the student and teacher networks. Both networks have the same architecture but different parameters. The output of the teacher network is centered with a mean computed over the batch. Each networks outputs a K dimensional feature that is normalized with a temperature softmax over the feature dimension. Their similarity is then measured with a cross-entropy loss. We apply a stop-gradient (sg) operator on the teacher to propagate gradients only through the student. The teacher parameters are updated with an exponential moving average (ema) of the student parameters.

$$c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i), \tag{4}$$

where $m > 0$ is a rate parameter and B is the batch size. Output sharpening is obtained by using a low value for the temperature τ_t in the teacher softmax normalization.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Table 7: **Important component for self-supervised ViT pre-training.** Models are trained for 300 epochs with ViT-S/16. We study the different components that matter for the k-NN and linear ("Lin.") evaluations. For the different variants, we highlight the differences from the default DINO setting. The best combination is the momentum encoder with the multicrop augmentation and the cross-entropy loss. We also report results with BYOL [30], MoCo-v2 [15] and SwAV [10].

	Method	Mom.	SK	MC	Loss	Pred.	k-NN	Lin.
1	DINO	✓	✗	✓	CE	✗	72.8	76.1
2		✗	✗	✓	CE	✗	0.1	0.1
3		✓	✓	✓	CE	✗	72.2	76.0
4		✓	✗	✗	CE	✗	67.9	72.5
5		✓	✗	✓	MSE	✗	52.6	62.4
6		✓	✗	✓	CE	✓	71.8	75.6
7	BYOL	✓	✗	✗	MSE	✓	66.6	71.4
8	MoCov2	✓	✗	✗	INCE	✗	62.0	71.6
9	SwAV	✗	✓	✓	CE	✗	64.7	71.8

SK: Sinkhorn-Knopp, MC: Multi-Crop, Pred.: Predictor
CE: Cross-Entropy, MSE: Mean Square Error, INCE: InfoNCE

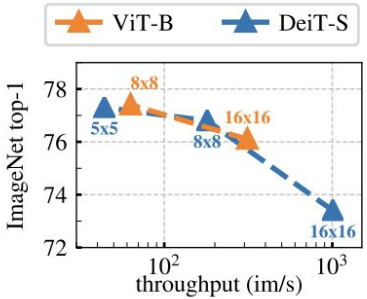
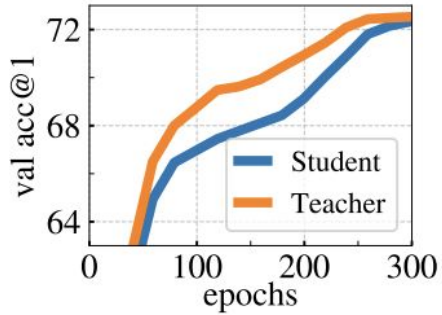


Figure 5: **Effect of Patch Size.** k-NN evaluation as a function of the throughputs for different input patch sizes with ViT-B and ViT-S. Models are trained for 300 epochs.



Teacher	Top-1
Student copy	0.1
Previous iter	0.1
Previous epoch	66.6
Momentum	72.8

Figure 6: Top-1 accuracy on ImageNet validation with k-NN classifier. **(left)** Comparison between the performance of the momentum teacher and the student during training. **(right)** Comparison between different types of teacher network. The momentum encoder leads to the best performance but is not the only viable option.

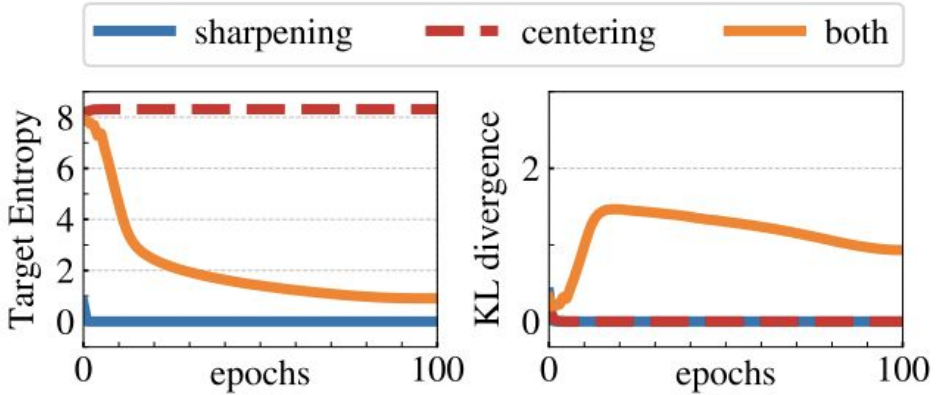
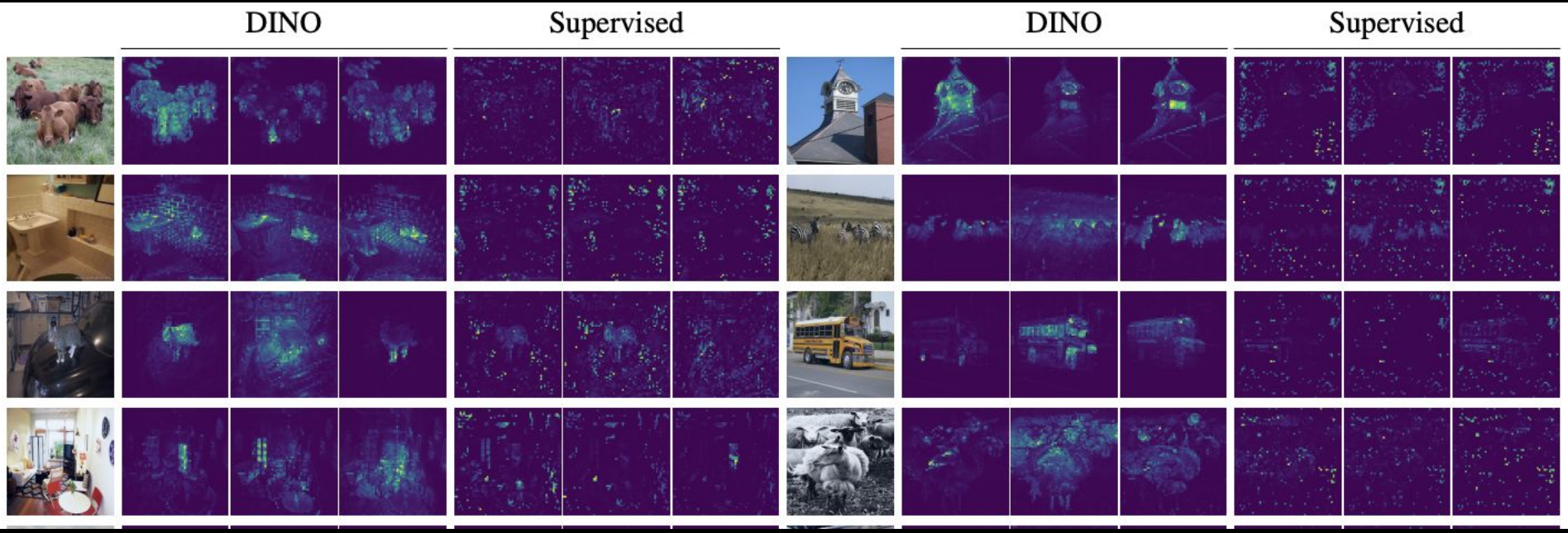


Figure 7: **Collapse study. (left)**: evolution of the teacher's target entropy along training epochs; **(right)**: evolution of KL divergence between teacher and student outputs.


DINO
Supervised
DINO
Supervised

DINOv2: Learning Robust Visual Features without Supervision

Oquab et al., 2023, # of citations: 13647

https://arxiv.org/abs/2304.07193

Abstract, Introduction

(1) Accelerating and stabilizing the training at scale
(2) Propose an automatic pipeline to build a dedicated, diverse, and curated image dataset

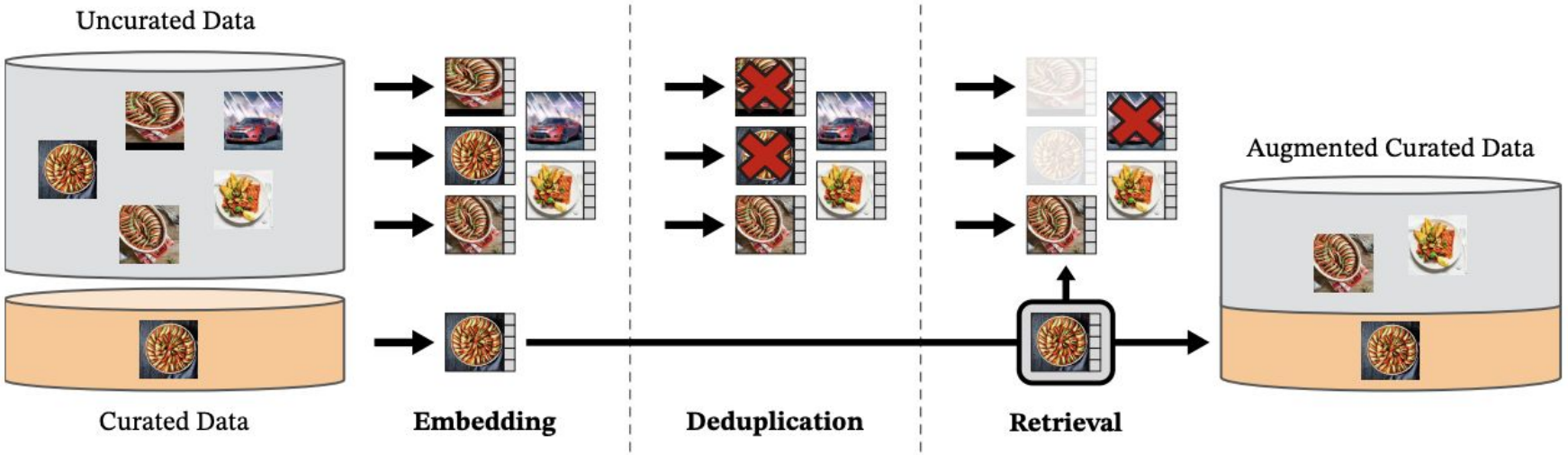


Figure 3: **Overview of our data processing pipeline.** Images from curated and uncurated data sources are first mapped to embeddings. Uncurated images are then deduplicated before being matched to curated images. The resulting combination augments the initial dataset through a self-supervised retrieval system.

New applications

(1) Patch-level objective
(2) Untying head weights between both objectives
(3) Sinkhorn-knopp centering
(4) KoLeo regularizer
(5) Adapting the resolution

New applications

$$\mathcal{L}_{iBOT} = -\sum_i p_{ti} \log p_{si}$$

$$\mathcal{L}_{DINO} = -\sum p_t \log p_s$$

- **KoLeo regularizer (Sablayrolles et al., 2019).** The KoLeo regularizer derives from the Kozachenko-Leonenko differential entropy estimator (see Beirlant et al. (1997); Delattre & Fournier (2017)) and encourages a uniform span of the features within a batch. Given a set of n vectors $(x_1, \ldots, x_n)$, it is defined as

$$\mathcal{L}_{\text{koleo}} = -\frac{1}{n}\sum_{i=1}^{n} \log(d_{n,i}),$$

where $d_{n,i} = \min_{j \neq i} \|x_i - x_j\|$ is the minimum distance between x_i and any other point within the batch. We also ℓ_2-normalize the features before computing this regularizer.

- **Sinkhorn-Knopp centering (Caron et al., 2020).** Ruan et al. (2023) recommend to replace the teacher softmax-centering step of DINO and iBot by the Sinkhorn-Knopp (SK) batch normalization of SwAV (Caron et al., 2020). We run the Sinkhorn-Knopp algorithm steps for 3 iterations. For the student, we apply the softmax normalization.

Ablation study

	INet-1k k-NN	INet-1k linear
iBOT	72.9	82.3
+(our reproduction)	74.5 ↑ 1.6	83.2 ↑ 0.9
+LayerScale, Stochastic Depth	75.4 ↑ 0.9	82.0 ↓ 1.2
+128k prototypes	76.6 ↑ 1.2	81.9 ↓ 0.1
+KoLeo	78.9 ↑ 2.3	82.5 ↑ 0.6
+SwiGLU FFN	78.7 ↓ 0.2	83.1 ↑ 0.6
+Patch size 14	78.9 ↑ 0.2	83.5 ↑ 0.4
+Teacher momentum 0.994	79.4 ↑ 0.5	83.6 ↑ 0.1
+Tweak warmup schedules	80.5 ↑ 1.1	83.8 ↑ 0.2
+Batch size 3k	81.7 ↑ 1.2	84.7 ↑ 0.9
+Sinkhorn-Knopp	81.7 =	84.7 =
+Untying heads = DINOv2	82.0 ↑ 0.3	84.5 ↓ 0.2

DINOv3

Siméoni et al., 2025, # of citations: 1775

https://arxiv.org/abs/2508.10104

Abstract, Introduction

(1) How do we collect useful data from unlabeled pool
   (a) Stratified k-means
   (b) DINOv2 curator
   (c) Open image datasets

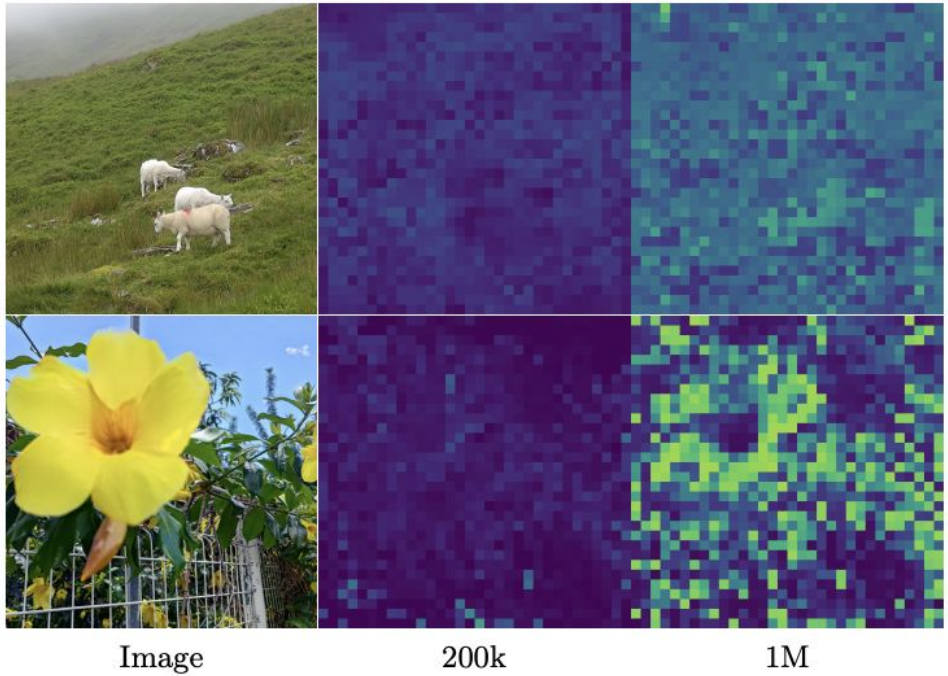
(2) How do we schedule lr and various hyperparams. w/o the knowledge of the end of the train
   (a) Remove all hyperparam. scheduling

(3) Feature representation degrades after initial train epochs → less scalable

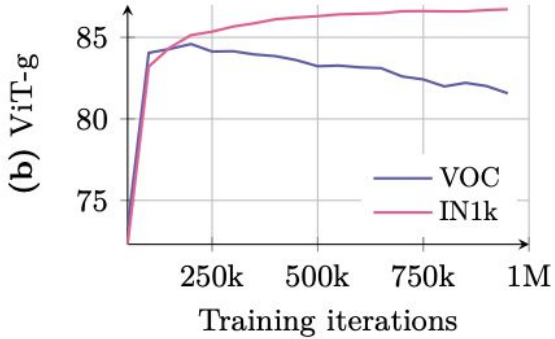
$$\mathcal{L}_{\text{Pre}} = \mathcal{L}_{\text{DINO}} + \mathcal{L}_{\text{iBOT}} + 0.1 * \mathcal{L}_{\text{DKoleo}}.$$

$$\mathcal{L}_{\text{Gram}} = \left\| \mathbf{X}_S \cdot \mathbf{X}_S^\top - \mathbf{X}_G \cdot \mathbf{X}_G^\top \right\|_{\text{F}}^2.$$

$$\mathcal{L}_{\text{Ref}} = w_{\text{D}}\mathcal{L}_{\text{DINO}} + \mathcal{L}_{\text{iBOT}} + w_{\text{DK}}\mathcal{L}_{\text{DKoleo}} + w_{\text{Gram}}\mathcal{L}_{\text{Gram}}.$$



(a) Cosine similarities between the CLS and output patches.



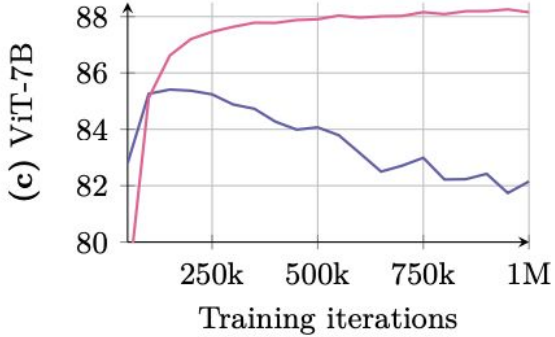


Figure 5: Evolution of the cosine similarities (a) and of the accuracy on ImageNet1k linear (IN1k) and segmentation on VOC for ViT-g (b) and ViT-7B (c). We observe that the segmentation performance is maximal when the cosine similarities between the patch tokens and the class tokens are low. As training progresses, these similarities increase and the performance on dense tasks decreases.

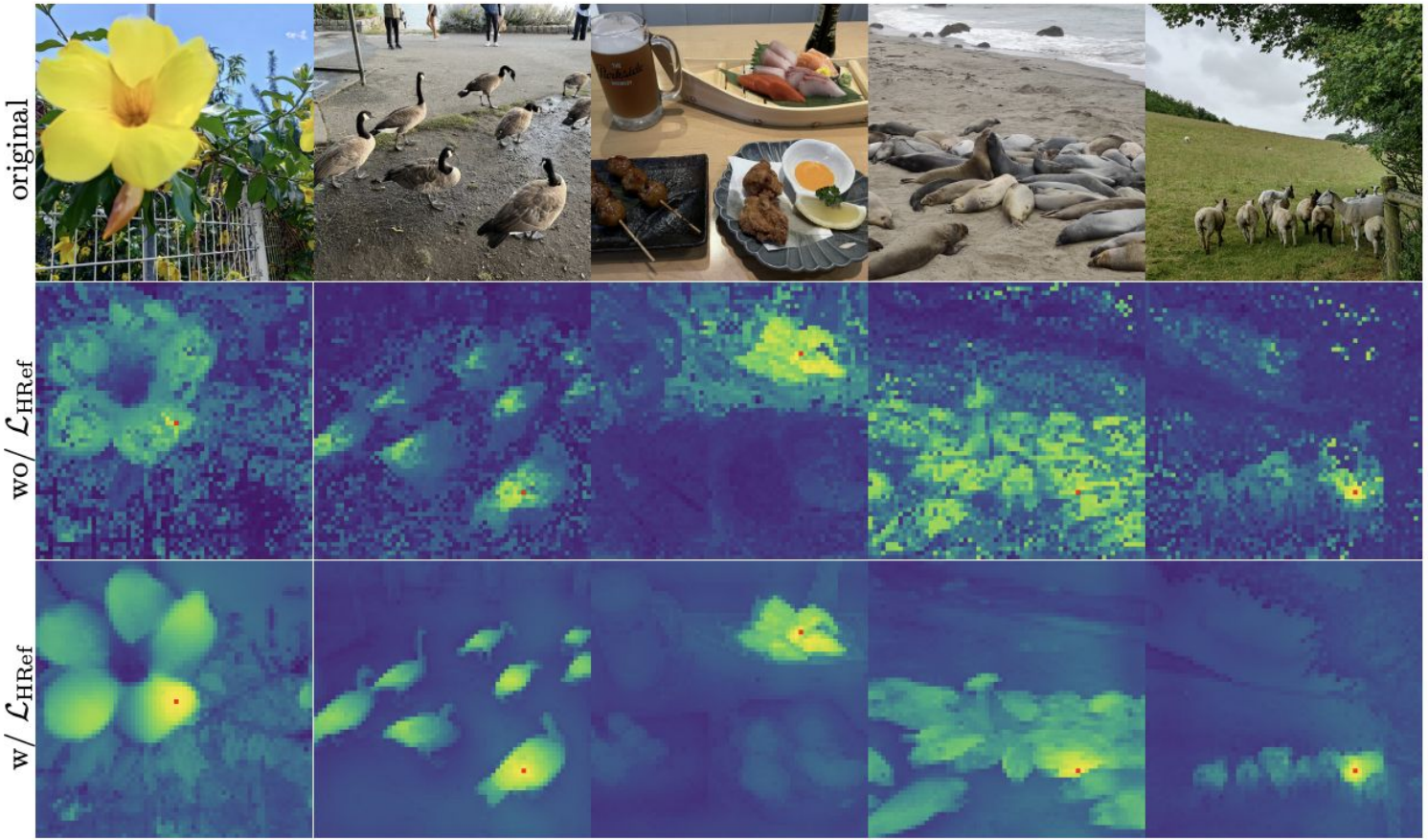


Figure 10: Qualitative effect of Gram anchoring. We visualize cosine maps before and after using the refinement objective $\mathcal{L}_{\text{HRef}}$. The input resolution of the images is 1024×1024 pixels.

ViT Recap (쉬어가는 시간)

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Dosovitskiy et al., 2020, # of citations: 112307

https://arxiv.org/abs/2010.11929

Abstract, Introduction

(1) Need MORE data

"Transformers lack some of the **inductive biases** inherent to CNNs, such as **translation equivariance** and **locality**, and therefore do not generalize well when trained on insufficient amounts of data."

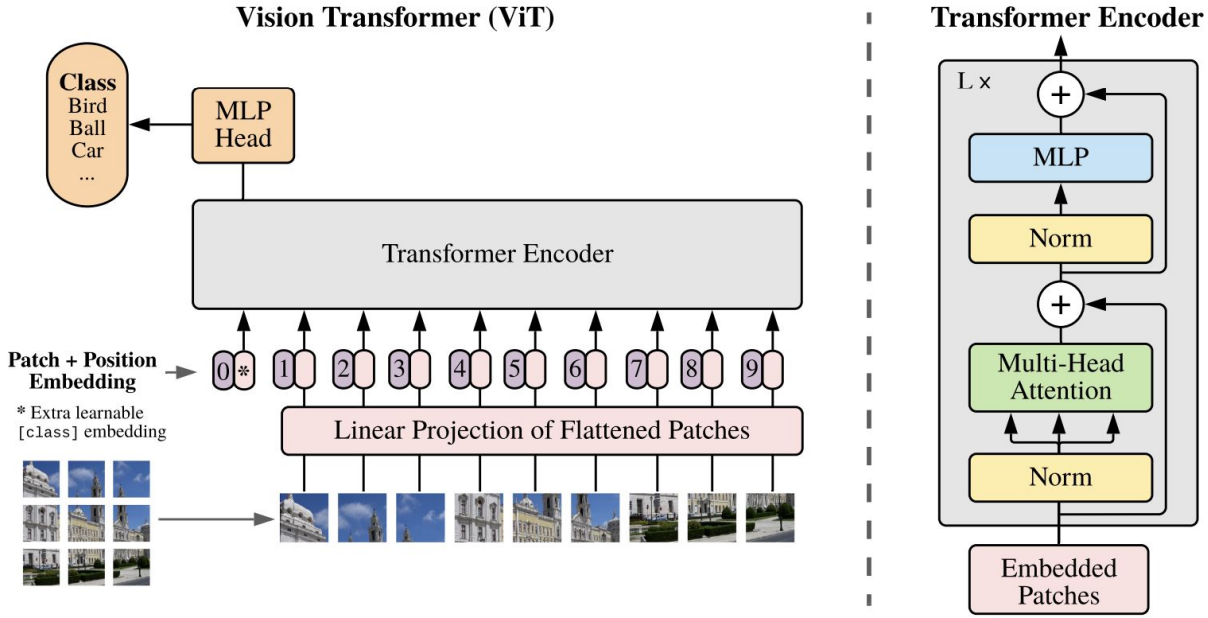


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

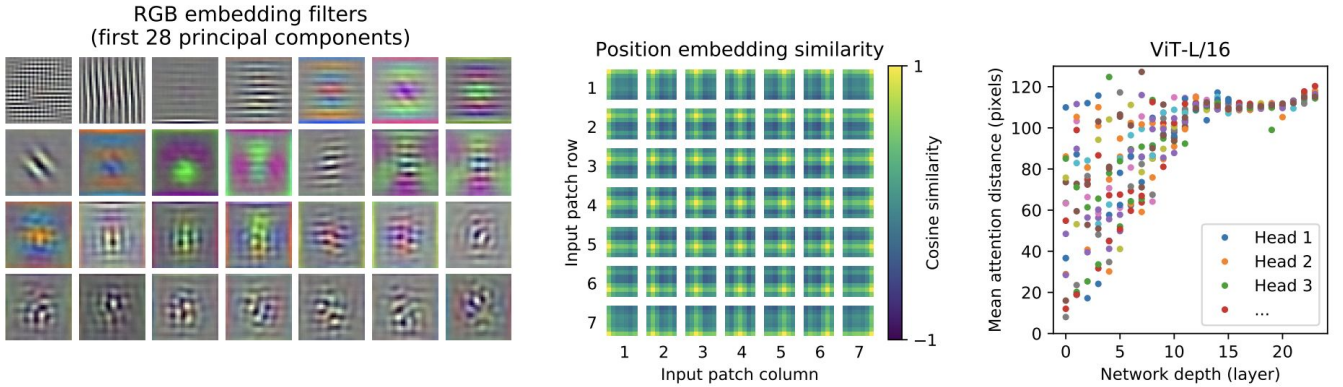


Figure 7: **Left:** Filters of the initial linear embedding of RGB values of ViT-L/32. **Center:** Similarity of position embeddings of ViT-L/32. Tiles show the cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches. **Right:** Size of attended area by head and network depth. Each dot shows the mean attention distance across images for one of 16 heads at one layer. See Appendix D.7 for details.

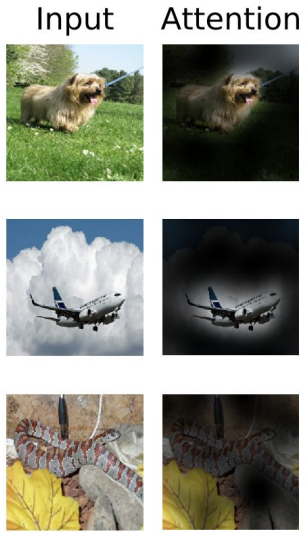


Figure 6: Representative examples of attention from the output token to the input space. See Appendix D.7 for details.

Training data-efficient image transformers & distillation through attention

Touvron et al., 2020, # of citations: 13538

https://arxiv.org/abs/2012.12877

Abstract, Introduction

(1) Distillation procedure based on a distillation token
(2) Image transformers learn more from a convnet than from another transformer with comparable performance
(3) Learned class and distillation tokens converge towards different vectors

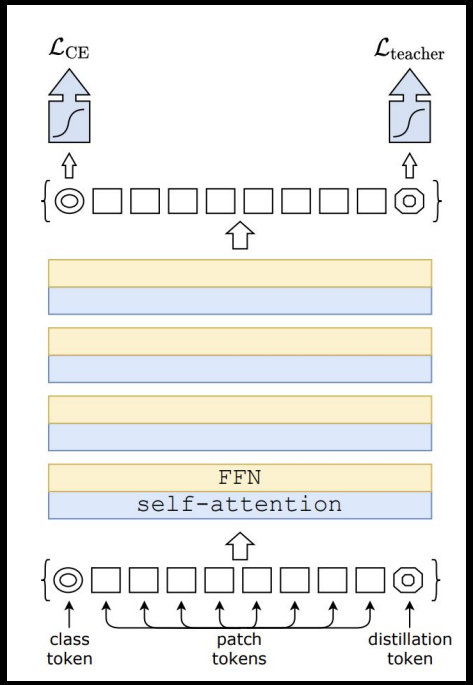
Soft distillation [24, 54] minimizes the Kullback-Leibler divergence between the softmax of the teacher and the softmax of the student model.

Let Z_t be the logits of the teacher model, Z_s the logits of the student model. We denote by τ the temperature for the distillation, λ the coefficient balancing the Kullback–Leibler divergence loss (KL) and the cross-entropy ($\mathcal{L}_{\text{CE}}$) on ground truth labels y, and ψ the softmax function. The distillation objective is

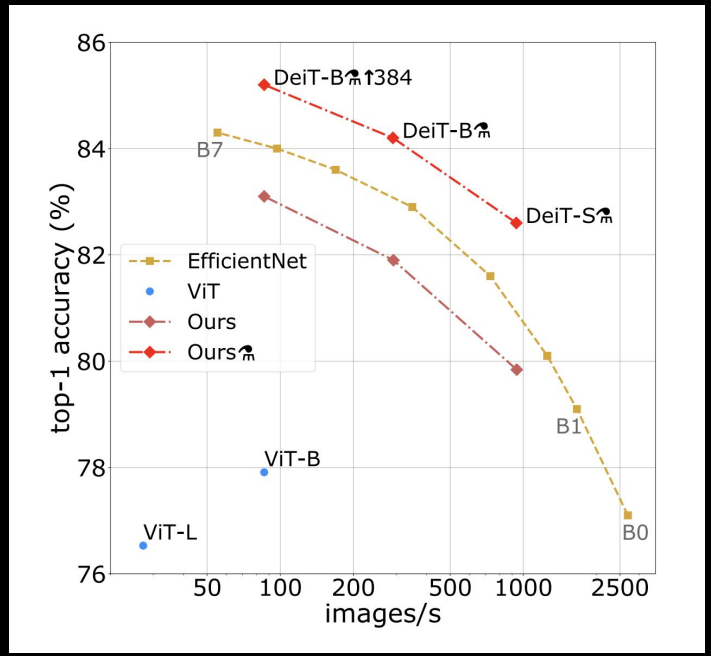
$$\mathcal{L}_{\text{global}} = (1-\lambda)\mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \lambda\tau^2 \text{KL}(\psi(Z_s/\tau), \psi(Z_t/\tau)). \quad (2)$$

Hard-label distillation. We introduce a variant of distillation where we take the hard decision of the teacher as a true label. Let $y_t = \text{argmax}_c Z_t(c)$ be the hard decision of the teacher, the objective associated with this hard-label distillation is:

$$\mathcal{L}_{\text{global}}^{\text{hardDistill}} = \frac{1}{2}\mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \frac{1}{2}\mathcal{L}_{\text{CE}}(\psi(Z_s), y_t). \quad (3)$$



method ↓	Supervision label	Supervision teacher	ImageNet top-1 (%) Ti 224	S 224	B 224	B↑384
DeiT– no distillation	✓	✗	72.2	79.8	81.8	83.1
DeiT– usual distillation	✗	soft	72.2	79.8	81.8	83.2
DeiT– hard distillation	✗	hard	74.3	80.9	83.0	84.0
DeiT⚗: class embedding	✓	hard	73.9	80.9	83.0	84.2
DeiT⚗: distil. embedding	✓	hard	74.6	81.1	83.1	84.4
DeiT⚗: class+distillation	✓	hard	74.5	81.2	83.4	84.5



Vision Transformers for Dense Prediction

Ranftl et al., 2021, # of citations: 4455

https://arxiv.org/abs/2103.13413

Abstract, Introduction

(1) Distillation procedure based on a distillation token
(2) Image transformers learn more from a convnet than from another transformer with comparable performance

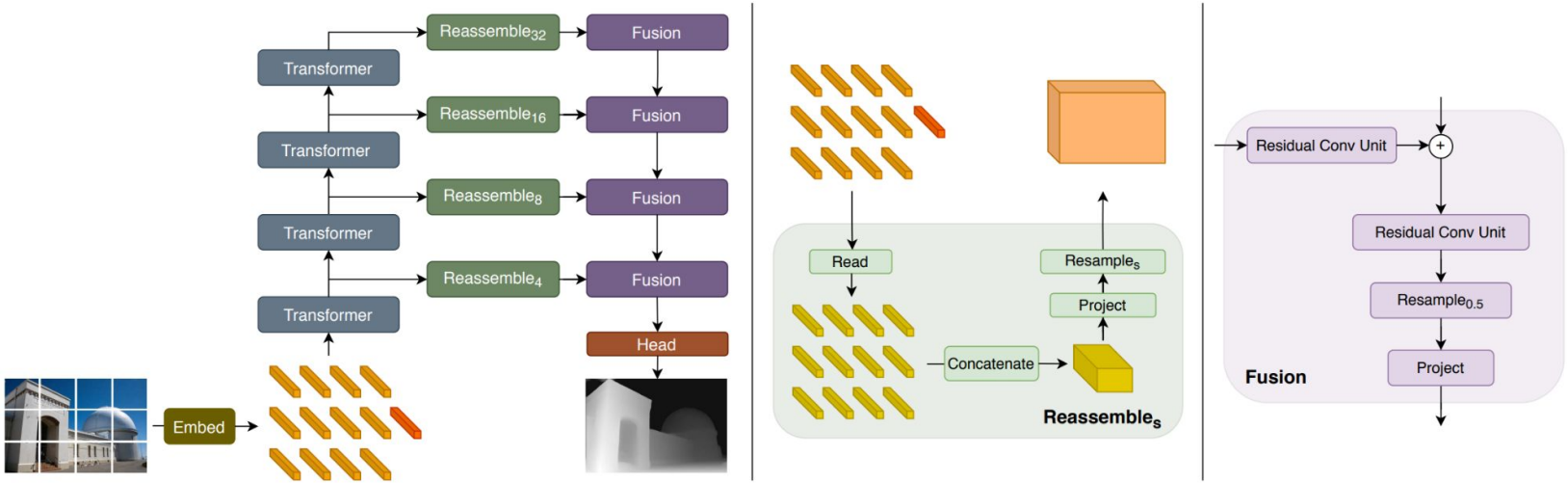


Figure 1. *Left*: Architecture overview. The input image is transformed into tokens (orange) either by extracting non-overlapping patches followed by a linear projection of their flattened representation (DPT-Base and DPT-Large) or by applying a ResNet-50 feature extractor (DPT-Hybrid). The image embedding is augmented with a positional embedding and a patch-independent readout token (red) is added. The tokens are passed through multiple transformer stages. We reassemble tokens from different stages into an image-like representation at multiple resolutions (green). Fusion modules (purple) progressively fuse and upsample the representations to generate a fine-grained prediction. *Center*: Overview of the Reassemble$_s$ operation. Tokens are assembled into feature maps with $\frac{1}{s}$ the spatial resolution of the input image. *Right*: Fusion blocks combine features using residual convolutional units [23] and upsample the feature maps.

$$\text{Read}_{ignore}(t) = \{t_1, \ldots, t_{N_p}\} \tag{2}$$

simply ignores the readout token,

$$\text{Read}_{add}(t) = \{t_1 + t_0, \ldots, t_{N_p} + t_0\} \tag{3}$$

passes the information from the readout token to all other tokens by adding the representations, and

$$\text{Read}_{proj}(t) = \{\text{mlp}(\text{cat}(t_1, t_0)), \ldots, \text{mlp}(\text{cat}(t_{N_p}, t_0))\} \tag{4}$$

passes information to the other tokens by concatenating the readout to all other tokens before projecting the representation to the original feature dimension D using a linear layer followed by a GELU non-linearity [17].

$$\text{Concatenate} : \mathbb{R}^{N_p \times D} \rightarrow \mathbb{R}^{\frac{H}{p} \times \frac{W}{p} \times D}. \tag{5}$$

We finally pass this representation to a spatial resampling layer that scales the representation to size $\frac{H}{s} \times \frac{W}{s}$ with $\hat{D}$ features per pixel:

$$\text{Resample}_s : \mathbb{R}^{\frac{H}{p} \times \frac{W}{p} \times D} \rightarrow \mathbb{R}^{\frac{H}{s} \times \frac{W}{s} \times \hat{D}}. \tag{6}$$

We implement this operation by first using 1×1 convolutions to project the input representation to $\hat{D}$, followed by a (strided) 3×3 convolution when $s \geq p$, or a strided 3×3 transpose convolution when $s < p$, to implement spatial downsampling and upsampling operations, respectively.